

Les Compréhensions en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Objectif de l'atelier

Les compréhensions sont l'une des fonctionnalités les plus distinctives de Python. Elles permettent de construire des collections et de traiter des flux de données de façon **concise, lisible et performante** —en une seule expression là où d'autres langages nécessitent plusieurs lignes.

Le problème sans compréhensions :

```
# Style impératif classique — verbeux, lent, bruyant
ventes = [50, 120, 300, 80, 150, 40]
```

```
ventes_importantes = []          # 1. créer une liste vide
for montant in ventes:          # 2. boucler
    if montant > 100:            # 3. filtrer
        ventes_importantes.append(montant) # 4. accumuler
```

Avec une compréhension de liste :

```
# La même logique en une ligne — lisible de gauche à droite
ventes_importantes = [montant for montant in ventes if montant > 100]
#                       ↑ quoi   ↑ source                       ↑ filtre
```

- **for montant in ventes** : source —on parcourt chaque élément
- **if montant > 100** : filtre —condition optionnelle, appliquée avant la transformation
- **montant** (en tête) : expression —ce qu'on met dans la liste résultante
- **gain réel** : pas de liste temporaire, pas de `.append()`, 30-50% plus rapide que la boucle équivalente en CPython

Vue d'ensemble —les 4 formes de compréhensions :

```
# 1. Compréhension de LISTE → [ ]
carres = [x**2 for x in range(5)]          # [0, 1, 4, 9, 16]

# 2. Compréhension de DICTIONNAIRE → { : }
index = {x: x**2 for x in range(5)}        # {0:0, 1:1, 2:4, 3:9, 4:16}

# 3. Compréhension de SET → { }
```

```

uniques = {x % 3 for x in range(10)}           # {0, 1, 2}

# 4. Expression GÉNÉRATRICE → ( ) ← pas de crochets !
somme = sum(x**2 for x in range(1_000_000))    # calculé sans liste en mémoire

```

- [] → liste —indexable, modifiable, tout en mémoire
- { : } → dict —accès par clé en O(1)
- { } → set —pas de doublons, non ordonné
- () → générateur — **paresseux** , produit les valeurs une à une, mémoire constante

2 □ Filtrage simple —Compréhension de liste

Problème métier

Sur une plateforme e-commerce, le département financier veut **isoler les ventes significatives** (> 100 TND) pour calculer les commissions des vendeurs. Les petites ventes sont exclues du calcul.

```

"""
Objectif : extraire les montants supérieurs à 100 TND.

Anatomie d'une compréhension de liste :
    [expression for élément in itérable if condition]
      ↑           ↑           ↑           ↑
    ce qu'on   variable   source de   filtre optionnel
    conserve  de boucle  données      (appliqué avant)
"""

```

```

ventes = [50, 120, 300, 80, 150, 40, 210, 95, 180]

```

```

# — Version compréhension
ventes_importantes = [montant for montant in ventes if montant > 100]

# — Équivalent explicite pour comprendre
ventes_importantes_v2 = []
for montant in ventes:
    if montant > 100:           # Le "if" dans la compréhension filtre ICI
        ventes_importantes_v2.append(montant)

print("Ventes brutes      :", ventes)
print("Ventes > 100 TND :", ventes_importantes)
print("Identical résultat :", ventes_importantes == ventes_importantes_v2)

```

```
# — Statistiques utiles sur le résultat —
total      = sum(ventes_importantes)
moyenne    = total / len(ventes_importantes)
nb_exclues = len(ventes) - len(ventes_importantes)

print(f"\n📊 Rapport :")
print(f" - Ventes totales analysées : {len(ventes)}")
print(f" - Ventes retenues (> 100) : {len(ventes_importantes)}")
print(f" - Ventes exclues           : {nb_exclues}")
print(f" - Montant total retenu       : {total} TND")
print(f" - Montant moyen retenu       : {moyenne:.2f} TND")

Ventes brutes      : [50, 120, 300, 80, 150, 40, 210, 95, 180]
Ventes > 100 TND   : [120, 300, 150, 210, 180]
Identical résultat : True
```

📊 Rapport :

- Ventes totales analysées : 9
- Ventes retenues (> 100) : 5
- Ventes exclues : 4
- Montant total retenu : 960 TND
- Montant moyen retenu : 192.00 TND

- **le filtre if est optionnel** — sans lui, tous les éléments sont inclus
 - **for montant in ventes** produit les éléments un par un — la liste originale n'est jamais modifiée
 - **ordre d'évaluation** : pour chaque montant, Python évalue d'abord le `if` — si `False`, l'expression de tête n'est pas évaluée du tout
 - **performance** : la compréhension est compilée en bytecode optimisé — `.append()` interne sans overhead Python
-

3 📦 Transformation avec logique métier

Problème métier

Le service commercial décide d'appliquer une **remise de 10%** sur toutes les commandes supérieures à 100 TND pour stimuler les gros acheteurs. Il faut produire la liste des montants après remise.

```
"""
```

Objectif : appliquer une transformation conditionnelle sur les données.

Règle métier :

- Si montant > 100 → montant × 0.9 (remise de 10%)
- Sinon → on ignore (filtre, pas de remplacement)

Structure utilisée :

```
[transformation(x) for x in source if condition(x)]
```

```
ventes = [50, 120, 300, 80, 150, 40, 210, 95, 180]
```

```
ventes_remise = [  
    round(montant * 0.9, 2)    # ← transformation : calcul de la remise  
    for montant in ventes  
    if montant > 100          # ← filtre : seulement les éligibles  
]
```

```
print("Ventes originales      :", ventes)  
print("Après remise 10%      :", ventes_remise)
```

```
# — Affichage comparatif avant/après —————  
print("\n Avant      →  Après remise")  
print(" " + "-" * 25)  
remise_paires = [  
    (m, round(m * 0.9, 2))  
    for m in ventes  
    if m > 100  
]  
for avant, apres in remise_paires:  
    economie = avant - apres  
    print(f" {avant:>6} TND → {apres:>6} TND (économie : {economie:.2f} TND)")  
  
total_avant = sum(m for m in ventes if m > 100)  
total_apres = sum(ventes_remise)  
print(f"\n Total avant remise : {total_avant} TND")  
print(f" Total après remise : {total_apres:.2f} TND")  
print(f" Remise totale      : {total_avant - total_apres:.2f} TND")  
  
Ventes originales      : [50, 120, 300, 80, 150, 40, 210, 95, 180]  
Après remise 10%      : [108.0, 270.0, 135.0, 189.0, 162.0]
```

```
Avant      →  Après remise  
-----  
120 TND → 108.0 TND (économie : 12.00 TND)  
300 TND → 270.0 TND (économie : 30.00 TND)  
150 TND → 135.0 TND (économie : 15.00 TND)  
210 TND → 189.0 TND (économie : 21.00 TND)
```

180 TND → 162.0 TND (économie : 18.00 TND)

Total avant remise : 960 TND
Total après remise : 864.00 TND
Remise totale : 96.00 TND

- `round(montant * 0.9, 2)` : l'expression de tête peut être **n'importe quelle expression Python** —appel de fonction, calcul, méthode de chaîne, etc.
- **l'expression est évaluée après le filtre** —`round(50 * 0.9, 2)` n'est jamais calculé pour les ventes ≤ 100
- `round(..., 2)` : indispensable pour les calculs monétaires —évite les flottants parasites comme 107.99999999

4 □ Expression conditionnelle dans une compréhension

Problème métier

Le rapport de direction doit classer **chaque vente** en catégorie, sans en exclure aucune. Cette fois, on ne filtre pas —on **transforme toutes les valeurs** avec un ternaire.

```
"""
Objectif : transformer chaque élément sans en exclure aucun.

Attention : deux usages du "if" dans une compréhension, très différents !

1. FILTRE (à la fin) : [x for x in data if condition]
   → exclut les éléments qui ne passent pas

2. TERNAIRE (en tête) : [A if condition else B for x in data]
   → transforme chaque élément, rien n'est exclu
"""

ventes = [50, 120, 300, 80, 150, 40, 210, 95, 180]

# — Classification binaire —
categories = [
    "Élevée" if montant > 100 else "Standard"
    for montant in ventes
]
```

```

# — Classification à 3 niveaux (ternaires imbriqués) —————
niveaux = [
    "Premium"   if montant >= 200 else
    "Élevée"    if montant >= 100 else
    "Standard"
    for montant in ventes
]

print("Ventes      :", ventes)
print("Catégories :", categories)
print("Niveaux     :", niveaux)

# — Tableau de synthèse —————
print("\n Montant | Catégorie | Niveau")
print(" " + "-" * 35)
for montant, cat, niv in zip(ventes, categories, niveaux):
    print(f" {montant:>6} TND | {cat:<10} | {niv}")

# — Comptage par niveau —————
from collections import Counter
compte = Counter(niveaux)
print(f"\n Répartition : {dict(compte)}")

Ventes      : [50, 120, 300, 80, 150, 40, 210, 95, 180]
Catégories  : ['Standard', 'Élevée', 'Élevée', 'Standard', 'Élevée', 'Standard', 'Élevée', 'Standard', 'Élevée']
Niveaux     : ['Standard', 'Élevée', 'Premium', 'Standard', 'Élevée', 'Standard', 'Premium', 'Standard', 'Premium']

```

Montant	Catégorie	Niveau
50 TND	Standard	Standard
120 TND	Élevée	Élevée
300 TND	Élevée	Premium
...		

▣ Répartition : {'Standard': 4, 'Élevée': 3, 'Premium': 2}

- **ternaire en tête** A if cond else B : **toujours** produit une valeur — longueur résultante = longueur source (contrairement au filtre)
- **ternaires imbriqués** : lisibles si formatés sur plusieurs lignes avec indentation —dès que la logique dépasse 3 niveaux, préférer une fonction
- **zip(ventes, categories, niveaux)** : parcourt plusieurs listes en parallèle —très idiomatique Python
- **Counter** de collections : compte les occurrences en une ligne —évite un dict + boucle manuelle

5 □ Compréhensions de dictionnaires

Problème métier

Le département marketing reçoit un catalogue de produits et doit le **normaliser, transformer et indexer** pour alimenter une API : clés standardisées, noms en majuscules, lookup rapide par SKU.

```
"""
Objectif : construire et transformer des dictionnaires par compréhension.

Syntaxe :
    {clé: valeur for élément in source if condition}

Avantage vs boucle for :
    - Plus rapide (bytecode optimisé)
    - Lisible d'un coup d'œil
    - Pas de dict vide + update manuels
"""

# — Transformation d'un catalogue brut —————
produits_bruts = [
    {"sku": "p-001", "nom": "clavier mécanique", "prix": 149.9, "stock": 12},
    {"sku": "p-002", "nom": "souris sans fil", "prix": 59.9, "stock": 45},
    {"sku": "p-003", "nom": "écran 27 pouces", "prix": 399.0, "stock": 3},
    {"sku": "p-004", "nom": "casque audio", "prix": 89.9, "stock": 0},
]

# Index SKU → nom normalisé
index_noms = {
    p["sku"].upper(): p["nom"].title()
    for p in produits_bruts
}

# Index SKU → prix avec TVA (19%)
index_prix_ttc = {
    p["sku"]: round(p["prix"] * 1.19, 2)
    for p in produits_bruts
}

# Filtrage : seulement les produits en stock
catalogue_disponible = {
    p["sku"]: p["nom"].title()
    for p in produits_bruts
    if p["stock"] > 0          # ← filtre : exclut les ruptures de stock
}
```

```

}

print("Index noms normalisés :", index_noms)
print("\nPrix TTC          :", index_prix_ttc)
print("\nDisponible en stock :", catalogue_disponible)

# — Accès direct O(1) —————
print(f"\nLookup 'p-001' → nom   : {index_noms['P-001']}")
print(f"Lookup 'p-003' → prix  : {index_prix_ttc['p-003']} TND TTC")

# — Inversion clé/valeur —————
# Utile pour retrouver le SKU depuis le nom
nom_vers_sku = {v: k for k, v in index_noms.items()}
print(f"\nLookup inverse 'Clavier Mécanique' → {nom_vers_sku['Clavier Mécanique']}")

Index noms normalisés : {'P-001': 'Clavier Mécanique', 'P-002': 'Souris Sans Fil', ...}

Prix TTC                : {'p-001': 178.38, 'p-002': 71.28, 'p-003': 474.81, 'p-004': 106.98}

Disponible en stock    : {'p-001': 'Clavier Mécanique', 'p-002': 'Souris Sans Fil', 'p-003': 'Écran 27 Pouces'}

Lookup 'p-001' → nom   : Clavier Mécanique
Lookup 'p-003' → prix  : 474.81 TND TTC

Lookup inverse 'Clavier Mécanique' → P-001

```

- **{clé: valeur for ... if ...}** : syntaxe complète —clé et valeur sont **deux expressions indépendantes**, chacune peut être aussi complexe que nécessaire
- **p["sku"].upper()** comme clé : normalisation à la construction —plus jamais de casse incohérente dans le dict résultant
- **inversion {v: k for k, v in d.items()} :** pattern classique pour le lookup inverse —attention si des valeurs se répètent (la dernière clé gagne)
- **accès dict[clé] en O(1) :** c'est tout l'intérêt de construire un index — une recherche dans une liste de 1000 produits est O(n), dans un dict O(1)

6 □ Compréhensions de sets

Problème métier

L'équipe data doit **dédupliquer** des catégories de produits, extraire les **codes pays uniques** des transactions et identifier les **IDs clients qui ont commandé plusieurs fois**.

"""

Objectif : construire des sets par compréhension pour des opérations ensemblistes.

Syntaxe :

{expression for élément in source if condition}
→ identique à la liste, mais { } au lieu de []

Propriétés du set :

- Pas de doublons (unicité garantie automatiquement)
- Non ordonné (l'ordre d'insertion n'est pas conservé)
- Opérations ensemblistes : union |, intersection &, différence -

"""

```
transactions = [
    {"id": 1, "client": "C001", "pays": "TN", "categorie": "informatique"},
    {"id": 2, "client": "C002", "pays": "FR", "categorie": "audio"},
    {"id": 3, "client": "C001", "pays": "TN", "categorie": "informatique"},
    {"id": 4, "client": "C003", "pays": "DE", "categorie": "peripheriques"},
    {"id": 5, "client": "C002", "pays": "FR", "categorie": "informatique"},
    {"id": 6, "client": "C004", "pays": "TN", "categorie": "audio"},
]

# — Valeurs uniques
pays_uniques = {t["pays"] for t in transactions}
categories_uniques = {t["categorie"] for t in transactions}
clients_uniques = {t["client"] for t in transactions}

print("Pays couverts      :", sorted(pays_uniques))
print("Catégories actives  :", sorted(categories_uniques))
print(f"Clients distincts   : {len(clients_uniques)} → {sorted(clients_uniques)}")

# — Opérations ensemblistes
clients_TN = {t["client"] for t in transactions if t["pays"] == "TN"}
clients_FR = {t["client"] for t in transactions if t["pays"] == "FR"}

print(f"\nClients TN      : {clients_TN}")
print(f"Clients FR      : {clients_FR}")
```

```

print(f"Clients TN ou FR      : {clients_TN | clients_FR}") # union
print(f"Clients TN et FR     : {clients_TN & clients_FR}") # intersection
print(f"Clients TN seulement : {clients_TN - clients_FR}") # différence

# — Déduplications avec ordre préservé —
# set ne préserve pas l'ordre → dict.fromkeys pour garder l'ordre d'apparition
categories_ordonnees = list(dict.fromkeys(t["categorie"] for t in transactions))
print(f"\nCatégories (ordre d'apparition) : {categories_ordonnees}")

Pays couverts      : ['DE', 'FR', 'TN']
Catégories actives : ['audio', 'informatique', 'peripheriques']
Clients distincts  : 4 → ['C001', 'C002', 'C003', 'C004']

Clients TN          : {'C001', 'C004'}
Clients FR          : {'C002'}
Clients TN ou FR    : {'C001', 'C002', 'C004'}
Clients TN et FR    : set()
Clients TN seulement : {'C001', 'C004'}

Catégories (ordre d'apparition) : ['informatique', 'audio', 'peripheriques']

• set par compréhension : la déduplication est automatique et gratuite
  —inutile de vérifier les doublons manuellement
• sorted(set_) : transforme le set en liste triée —nécessaire à chaque fois
  qu'on a besoin d'un ordre stable
• opérations ensemblistes (|, &, -) : expressives et efficaces —bien plus
  lisibles que des boucles imbriquées
• dict.fromkeys(itérable) : astuce Python pour dédupliquer en
préservant l'ordre —disponible depuis Python 3.7 où les dicts sont
  ordonnés

```

7 □ Générateurs —traitement de gros volumes

Contexte métier

La plateforme reçoit **10 millions de transactions par jour**. Les charger toutes en mémoire avec une liste provoquerait un crash ou une lenteur inacceptable. Les **générateurs** résolvent ce problème en produisant les valeurs **une à une, à la demande**.

```

"""
Objectif : traiter un flux massif sans saturer la mémoire.

```

Principe du générateur :

- Une fonction avec "yield" au lieu de "return"
- Suspend son exécution à chaque "yield" et reprend là où elle s'est arrêtée
- Ne calcule que ce qu'on lui demande (évaluation paresseuse)
- Mémoire constante quelle que soit la taille des données

Différence fondamentale :

list → tout en mémoire dès le départ → RAM proportionnelle à la taille
gen → état interne seulement → RAM constante (quelques dizaines d'octets)
"""

```
import sys
import time
```

```
# — Simulation d'un flux de transactions —————
def flux_transactions(nb: int):
    """Génère nb transactions simulées — une à la fois."""
    for i in range(nb):
        yield {
            "id": i,
            "montant": (i * 37) % 500 + 10, # montant pseudo-aléatoire 10..509
            "client": f"C{i % 1000:04d}",
            "statut": "OK" if i % 7 != 0 else "KO"
        }
```

```
# — Mesure mémoire : Liste vs générateur —————
N = 100_000

liste_tx = [{"id": i, "montant": i} for i in range(N)]
gen_tx = ({ "id": i, "montant": i } for i in range(N))

print(f"Liste de {N} éléments : {sys.getsizeof(liste_tx):>10,} octets")
print(f"Générateur équivalent : {sys.getsizeof(gen_tx):>10,} octets")
print(f"Ratio mémoire : {sys.getsizeof(liste_tx) // sys.getsizeof(gen_tx):,}>10,}x")
```

```
# — Consommation progressive avec next() —————
gen = flux_transactions(10_000_000) # 10 millions de tx — aucune RAM utilisée

print(f"\nType de l'objet : {type(gen)}")
print("\nPremières transactions (consommation à la demande) :")
for _ in range(3):
    tx = next(gen) # produit UNE transaction, pas toutes
    print(f" {tx}")
```

```
# — Traitement d'un gros flux sans liste intermédiaire —————
t0 = time.perf_counter()
```

```
total_ok = sum(1 for tx in flux_transactions(1_000_000) if tx["statut"] == "OK")
total_ko = sum(1 for tx in flux_transactions(1_000_000) if tx["statut"] == "KO")
montant = sum(tx["montant"] for tx in flux_transactions(1_000_000) if tx["statut"] == "OK")

duree = time.perf_counter() - t0
```

```
print(f"\n🔍 Analyse de 1 000 000 transactions :")
print(f" - Transactions OK : {total_ok:,}")
print(f" - Transactions KO : {total_ko:,}")
print(f" - Montant total OK : {montant:,} TND")
print(f" - Durée : {duree:.2f}s — sans jamais stocker 1M de dicts")
```

```
Liste de 100 000 éléments : 800,984 octets
Générateur équivalent : 120 octets
Ratio mémoire : 6,674x
```

Type de l'objet : <class 'generator'>

Premières transactions :

```
{'id': 0, 'montant': 10, 'client': 'C0000', 'statut': 'OK'}
{'id': 1, 'montant': 47, 'client': 'C0001', 'statut': 'OK'}
{'id': 2, 'montant': 84, 'client': 'C0002', 'statut': 'OK'}
```

🔍 Analyse de 1 000 000 transactions :

```
- Transactions OK : 857,143
- Transactions KO : 142,857
- Montant total OK : 214,869,440 TND
- Durée : 0.87s — sans jamais stocker 1M de dicts
```

- **yield** : suspend la fonction et **retourne** une valeur —la prochaine fois que `next()` est appelé, la fonction reprend exactement là où elle s'est arrêtée
- **next(gen)** : consomme une seule valeur —Python appelle `__next__()` sur le générateur
- **120 octets vs 800 000 octets** : le générateur ne stocke que son état interne (pointeur de ligne, variables locales) —pas les données
- **StopIteration** : exception levée quand le générateur est épuisé —for la capture automatiquement, `next()` la propage

8 📄 Liste vs générateur —comparaison approfondie

"""

Objectif : comparer les comportements de liste et générateur sur tous les axes.

Axes de comparaison :

- 1. Mémoire*
- 2. Vitesse de construction*
- 3. Accès par index*
- 4. Réutilisabilité*
- 5. Consommation partielle*

"""

```
import sys
import time
```

```
N = 500_000
```

```
# — 1. Mémoire —————
liste = [i * 2 for i in range(N)]
gen    = (i * 2 for i in range(N))

print(f"Mémoire liste      : {sys.getsizeof(liste):,} octets")
print(f"Mémoire générateur : {sys.getsizeof(gen)} octets\n")

# — 2. Construction —————
t0 = time.perf_counter()
liste2 = [i * 2 for i in range(N)]
duree_liste = time.perf_counter() - t0

t0 = time.perf_counter()
gen2 = (i * 2 for i in range(N)) # instantané — rien n'est calculé encore
duree_gen = time.perf_counter() - t0

print(f"Construction liste      : {duree_liste*1000:.2f} ms")
print(f"Construction générateur : {duree_gen*1000:.4f} ms (← quasi-instantané)\n")

# — 3. Accès par index —————
print(f"liste[42] = {liste[42]}") # ☑ accès direct O(1)
try:
    gen[42] # ☑ Les générateurs ne supportent pas []
except TypeError as e:
    print(f"gen[42] → {e}\n")

# — 4. Réutilisabilité —————
somme1 = sum(liste) # ☑ peut être parcouru plusieurs fois
somme2 = sum(liste) # ☑ toujours disponible
print(f"Liste parcourue 2 fois : {somme1} == {somme2}\n")
```

```

gen3 = (i for i in range(5))
print(f"Générateur 1ère fois : {list(gen3)}") # [0, 1, 2, 3, 4]
print(f"Générateur 2ème fois : {list(gen3)}\n") # [] ← épuisé !

# — 5. Consommation partielle —————
import itertools

gen4 = (i**2 for i in range(1_000_000))
# On prend seulement les 5 premiers — Les 999 995 autres ne sont jamais calculés
premiers = list(itertools.islice(gen4, 5))
print(f"5 premiers carrés (sans calculer les 999 995 suivants) : {premiers}")

# — Tableau récapitulatif —————
print("""

```

Critère	Liste []	Générateur ()
Mémoire	Proportionnelle	Constante (~120 o)
Construction	Immédiate	Instantanée (paresse)
Accès par index	❑ O(1)	❑ impossible
Réutilisable	❑ oui	❑ épuisé après
Consommation part.	❑ oui	❑ oui (islice)
Gros volumes	❑ RAM limitante	❑ idéal

```

""")

```

- **liste** : toutes les valeurs calculées et stockées immédiatement —idéal quand on a besoin d'accès aléatoire ou de parcours multiples
- **générateur** : rien n'est calculé avant d'être demandé —idéal pour les flux, les fichiers, les APIs paginées
- **itertools.islice(gen, n)** : consomme exactement n valeurs d'un générateur —les suivantes ne sont jamais calculées
- **générateur épuisé** : après le premier parcours complet, `list(gen)` retourne `[]` —il faut **recréer** le générateur (rappeler la fonction)

9 □ Compréhensions imbriquées

Problème métier

Le système de commandes stocke les articles dans des paniers imbriqués. Il faut **aplatir**, **croiser** et **reconstruire** des structures à plusieurs niveaux.

```

"""

```

Objectif : maîtriser les compréhensions à plusieurs niveaux de boucle.

Règle de Lecture :

```
[expression for x in A for y in B if condition]
↓ équivalent à :
for x in A:
    for y in B:
        if condition:
            résultat.append(expression)
```

Attention : L'ordre des "for" détermine l'ordre des résultats.
"""

— Cas 1 : aplatir des paniers de commandes

```
paniers = [
    {"client": "C001", "articles": ["clavier", "souris", "tapis"]},
    {"client": "C002", "articles": ["écran"]},
    {"client": "C003", "articles": ["casque", "micro"]},
]

# Extraire tous les articles en une liste plate
tous_articles = [
    article
    for panier in paniers          # niveau 1 : parcourir les paniers
    for article in panier["articles"] # niveau 2 : parcourir chaque panier
]
print("Tous les articles :", tous_articles)

# Avec filtre : articles de plus de 6 caractères
articles_long = [
    article
    for panier in paniers
    for article in panier["articles"]
    if len(article) > 6
]
print("Articles (nom long) :", articles_long)

# — Cas 2 : produit cartésien — toutes les combinaisons tailles/couleurs —
tailles = ["S", "M", "L", "XL"]
couleurs = ["noir", "blanc", "rouge"]

variantes = [
    f"{taille}-{couleur}"
    for taille in tailles
    for couleur in couleurs
]
```

```

print(f"\nVariantes produit ({len(variantes)}) :", variantes)

# — Cas 3 : matrice de scores de similarité —————
produits = ["clavier", "souris", "écran"]

# Toutes les paires distinctes (pas de (A,A), pas de (B,A) si (A,B) existe)
paires = [
    (p1, p2)
    for i, p1 in enumerate(produits)
    for j, p2 in enumerate(produits)
    if i < j          # ← filtre : évite les doublons et les auto-paires
]
print(f"\nPaires de produits : {paires}")

# — Cas 4 : matrice 2D —————
# Bonne pratique : list of lists par compréhension
n, m = 3, 4
matrice = [
    [i * m + j for j in range(m)] # ← compréhension interne = une ligne
    for i in range(n)             # ← compréhension externe = les lignes
]
print("\nMatrice 3x4 :")
for ligne in matrice:
    print(" ", ligne)

Tous les articles : ['clavier', 'souris', 'tapis', 'écran', 'casque', 'micro']
Articles (nom long) : ['clavier', 'souris', 'casque']

Variantes produit (12) : ['S-noir', 'S-blanc', 'S-rouge', 'M-noir', ...]

Paires de produits : [('clavier', 'souris'), ('clavier', 'écran'), ('souris', 'écran')]

Matrice 3x4 :
[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]

```

- **ordre des for** : le premier for est la boucle **externe** —l'ordre des résultats en dépend directement
- **if i < j** : filtre sur les indices pour générer uniquement les paires uniques —évite (A,B) et (B,A) en même temps
- **compréhension imbriquée pour matrice** : [... for j in ...] for i in ...] —la compréhension interne produit une ligne, l'externe produit les lignes
- **règle des 2 niveaux** : au-delà de 2 for imbriqués, la lisibilité chute —décomposer en fonctions ou en plusieurs étapes

□ Bonnes pratiques

Les compréhensions peuvent devenir un piège de lisibilité si mal utilisées. Voici les règles issues de l'expérience terrain.

```
"""
Règle 1 — Préférer la lisibilité à la concision.
Règle 2 — Extraire la logique complexe dans une fonction.
Règle 3 — Utiliser un générateur pour les gros volumes (sum, any, all, for).
Règle 4 — Ne jamais dépasser 2 niveaux d'imbrication.
Règle 5 — Pas d'effets de bord dans une compréhension.
"""

# ☹ MAUVAIS : trop compact, illisible, avec effet de bord
log = []
result = [log.append(x) or x*2 for x in range(5) if x % 2 == 0 if x > 0]

# ☺ BON : Logique extraite, intention claire
def est_eligible(x):
    return x % 2 == 0 and x > 0

def transformer(x):
    return x * 2

result_propre = [transformer(x) for x in range(5) if est_eligible(x)]
print("Résultat propre :", result_propre) # [4, 8]

# ☹ MAUVAIS : compréhension pour un seul résultat agrégé → utiliser sum()
total_mauvais = sum([x**2 for x in range(1_000_000)]) # liste créée inutilement

# ☺ BON : expression génératrice directement dans sum()
total_bon = sum(x**2 for x in range(1_000_000)) # pas de liste, même résultat
print("Totaux égaux :", total_mauvais == total_bon)

# — Cas où la boucle for est MEILLEURE qu'une compréhension —————
commandes = [
    {"id": 1, "client": "C1", "items": ["A", "B"]},
    {"id": 2, "client": "C2", "items": ["C"]},
]

# ☹ compréhension avec effet de bord (mutation d'un dict externe)
```

```

index = {}
_ = [index.update({c["id"]: c["client"]}) for c in commandes]  # à éviter !

# ☞ boucle for explicite quand le but est un effet de bord
index_propre = {}
for c in commandes:
    index_propre[c["id"]] = c["client"]

# ☞ ENCORE MIEUX : compréhension de dict (l'outil prévu pour ça)
index_ideal = {c["id"]: c["client"] for c in commandes}

print("Index :", index_ideal)

# — Guide de décision —————
print("""
Quand utiliser quoi :

[x for x in data]           → liste, accès multiple, besoin d'index
{k: v for ...}              → dict, lookup rapide, transformation
{x for x in data}           → set, déduplication, opérations ensemblistes
(x for x in data)           → générateur, gros volume, sum/any/all/for
sum/any/all(x for x in data) → agrégation sans liste intermédiaire
boucle for explicite        → effets de bord, logique complexe, lisibilité
""")

```

- **effets de bord** dans une compréhension (append, update, print) : à **bannir** —une compréhension doit toujours **produire** une valeur, jamais modifier l'état externe
- **sum(gen) sans crochets** : supprimer les [] internes épargne la liste intermédiaire —même résultat, moitié moins de mémoire
- **any(cond for x in data)** et **all(cond for x in data)** : court-circuitent dès le premier résultat —encore plus efficaces qu'une compréhension de liste + any(liste)
- **test de la règle des 2 niveaux** : si vous ne pouvez pas lire la compréhension à voix haute en une phrase naturelle, il faut la décomposer

☐ Récapitulatif —choisir la bonne forme

[...] liste → accès par index, parcours multiples, petits/moyens volumes · {k:v} dict → lookup O(1), transformation de structure · {...} set → déduplication, intersection/union · (...) générateur → flux, gros volumes, agrégation unique · **boucle for** → effets de bord, logique métier complexe, plusieurs mutations simultanées

20 questions intelligentes — Compréhensions et générateurs (Python)

1) Pourquoi un générateur affiche un objet au lieu des valeurs ?

Créer le fichier **q01_générateur_affichage.py** :

```
"""
Objectif
Comprendre pourquoi un générateur n'affiche pas ses valeurs directement.

Idée
- Un générateur est un objet paresseux.
- Il produit des valeurs uniquement quand on le consomme (for, next, list, sum...).
- Afficher Le générateur affiche l'objet, pas son contenu.
"""

def multiples_de_trois(valeurs):
    for v in valeurs:
        if v % 3 == 0:
            yield v

data = range(1, 21)

gen = multiples_de_trois(data)

print("Affichage direct du générateur :", gen)

print("Consommation avec next :")
print(next(gen))
print(next(gen))

print("Consommation du reste avec for :")
for x in gen:
    print(x)
```

2) Pourquoi list(gen) vide le générateur ?

Créer le fichier **q02_list_epuise_générateur.py** :

```
"""
Objectif
Montrer qu'un générateur ne peut pas être consommé deux fois sans le recréer.
```

Idée

- Un générateur garde un état interne.
 - Une fois parcouru, il est épuisé.
 - Pour recommencer, il faut créer un nouveau générateur.
- """

```
def nombres_paires(n):  
    for i in range(n):  
        if i % 2 == 0:  
            yield i
```

```
gen = nombres_paires(10)
```

```
premiere_fois = list(gen)  
deuxieme_fois = list(gen)
```

```
print("Première consommation :", premiere_fois)  
print("Deuxième consommation :", deuxieme_fois)
```

```
gen2 = nombres_paires(10)  
print("Recréation d'un générateur :", list(gen2))
```

3) Différence pratique entre compréhension de liste et expression génératrice

Créer le fichier **q03_liste_vs_generateur.py** :

"""

Objectif

Comparer résultat, type et consommation mémoire (niveau intuition).

Idée

- Compréhension de liste : construit tout de suite.
 - Expression génératrice : produit à la demande.
- """

```
import sys
```

```
data = range(100000)
```

```
liste = [x * 2 for x in data]  
gen = (x * 2 for x in data)
```

```

print("Type liste :", type(liste))
print("Type générateur :", type(gen))

print("Taille liste :", sys.getsizeof(liste))
print("Taille générateur :", sys.getsizeof(gen))

print("Exemple valeurs (liste) :", liste[0], liste[1], liste[2])
print("Exemple valeurs (générateur) :", next(gen), next(gen), next(gen))

```

4) Pourquoi une compréhension est souvent plus rapide qu'un for + append ?

Créer le fichier **q04_perf_listcomp.py** :

```

"""
Objectif
Montrer une mesure simple : Liste via for vs compréhension.

Idée
- La compréhension est optimisée côté CPython (boucle interne plus rapide).
- On mesure sur une taille raisonnable.
"""

import time

n = 500000

t0 = time.perf_counter()
res1 = []
for i in range(n):
    if i % 3 == 0:
        res1.append(i * 2)
t1 = time.perf_counter()

t2 = time.perf_counter()
res2 = [i * 2 for i in range(n) if i % 3 == 0]
t3 = time.perf_counter()

print("Même résultat :", res1[:5] == res2[:5] and len(res1) == len(res2))
print("for+append :", round(t1 - t0, 4), "s")
print("listcomp  :", round(t3 - t2, 4), "s")

```

5) Comment appliquer une transformation conditionnelle dans une seule compréhension ?

Créer le fichier `q05_ternaire_comprehension.py` :

```
"""
Objectif
Utiliser une expression conditionnelle (if/else) dans la partie expression.

Différence importante
- if filtre : placé à la fin
- if/else transforme : placé au début (expression)
"""

ventes = [50, 120, 300, 80, 150, 40]

etiquettes = ["VIP" if v > 100 else "STANDARD" for v in ventes]

print("Ventes :", ventes)
print("Étiquettes :", etiquettes)
```

6) Pourquoi ce code ne filtre pas comme prévu ?

Créer le fichier `q06_if_filtre_vs_if_else.py` :

```
"""
Objectif
Montrer la différence entre filtrer et transformer.

Cas
- Filtrer : on supprime des éléments
- Transformer : on garde tout mais on change la valeur
"""

data = [1, 2, 3, 4, 5]

filtre = [x for x in data if x % 2 == 0]
transforme = [x if x % 2 == 0 else 0 for x in data]

print("Original :", data)
print("Filtré (pairs) :", filtre)
print("Transformé (impairs -> 0) :", transforme)
```

7) Comment éviter une compréhension illisible ?

Créer le fichier **q07_lisible_ou_pas.py** :

```
"""
Objectif
Montrer une règle pratique : dès que la logique devient complexe, on découpe.

Démarche
- Version compacte (peu lisible)
- Version découpée (lisible)
"""

commandes = [
    {"id": 1, "total": 120, "client": "A"},
    {"id": 2, "total": 80, "client": "B"},
    {"id": 3, "total": 300, "client": "A"},
    {"id": 4, "total": 150, "client": "C"},
]

# Version compacte
ids_client_a_vip = [c["id"] for c in commandes if c["client"] == "A" and c["total"] > 100]

# Version lisible
def est_vip_commande(cmd):
    return cmd["client"] == "A" and cmd["total"] > 100

ids_lisibles = [c["id"] for c in commandes if est_vip_commande(c)]

print("Compact :", ids_client_a_vip)
print("Lisible :", ids_lisibles)
```

8) Pourquoi utiliser une compréhension de dictionnaire plutôt qu'un for ?

Créer le fichier **q08_dict_comprehension.py** :

```
"""
Objectif
Transformer une structure de données en dictionnaire.

Cas métier
Indexation d'un catalogue par code produit, avec normalisation du nom.
"""
```

```

produits = [
    {"sku": "P100", "nom": "clavier"},
    {"sku": "P200", "nom": "souris"},
    {"sku": "P300", "nom": "ecran"},
]

index = {p["sku"]: p["nom"].upper() for p in produits}

print("Index catalogue :", index)
print("Accès direct P200 :", index["P200"])

```

9) Comment inverser un dictionnaire (valeur -> clé) sans bug ?

Créer le fichier **q09_inverser_dict.py** :

```

"""
Objectif
Inverser un dictionnaire proprement.

Piège
Si deux clés ont la même valeur, la dernière écrase la précédente.

Démarche
1) Inversion directe
2) Version robuste : regrouper les clés par valeur
"""

roles = {"u1": "admin", "u2": "user", "u3": "user"}

inverse_simple = {v: k for k, v in roles.items()}
print("Inverse simple :", inverse_simple)

inverse_robuste = {}
for user, role in roles.items():
    inverse_robuste.setdefault(role, []).append(user)

print("Inverse robuste :", inverse_robuste)

```

10) Pourquoi set comprehension ne garde pas l'ordre ?

Créer le fichier **q10_set_sans_ordre.py** :


```

"""
Objectif
Montrer qu'un set ne garantit pas l'ordre d'affichage.

Cas
Utiliser un set pour dédupliquer, puis trier si besoin d'un ordre stable.
"""

```

```

data = [5, 3, 3, 1, 2, 5, 4, 2]

unique = {x for x in data}
unique_trie = sorted(unique)

print("Original :", data)
print("Unique (set) :", unique)
print("Unique trié :", unique_trie)

```

11) Comment faire une matrice correctement, sans piège de duplication de lignes ?

Créer le fichier **q11_matrice_correcte.py** :

```

"""
Objectif
Construire une matrice 3x3 sans partager les mêmes sous-listes.

Piège classique
[[0]*3]*3 crée 3 références vers la même ligne.

Solution
Créer chaque ligne indépendamment via une compréhension.
"""

```

```

mauvaise = [[0] * 3] * 3
mauvaise[0][0] = 9

bonne = [[0] * 3 for _ in range(3)]
bonne[0][0] = 9

print("Mauvaise matrice :", mauvaise)
print("Bonne matrice :", bonne)

```

12) Comment aplatir une liste de listes (flatten) proprement ?

Créer le fichier **q12_flatten.py** :

```
"""
Objectif
Aplatir une liste de listes.

Technique
Compréhension imbriquée : for ligne puis for valeur.
"""

paniers = [
    ["clavier", "souris"],
    ["ecran"],
    ["casque", "micro"]
]

flat = [item for panier in paniers for item in panier]

print("Original :", paniers)
print("Aplati :", flat)
```

13) Pourquoi l'ordre des for change le résultat ?

Créer le fichier **q13_ordre_for.py** :

```
"""
Objectif
Montrer que l'ordre des boucles dans une compréhension imbriquée est déterminant.

Règle
Lecture de gauche à droite :
- for i ...
- puis for j ...
"""

a = [(i, j) for i in range(2) for j in range(3)]
b = [(i, j) for j in range(3) for i in range(2)]

print("Ordre i puis j :", a)
print("Ordre j puis i :", b)
```

14) Comment filtrer avec plusieurs conditions sans rendre le code fragile ?

Créer le fichier **q14_multi_conditions.py** :

```
"""
```

Objectif

Filtrer avec plusieurs conditions.

Bonne pratique

Regrouper les règles dans une fonction nommée si la condition devient longue.

```
"""
```

```
transactions = [
    {"id": 1, "montant": 120, "pays": "TN", "statut": "OK"},
    {"id": 2, "montant": 90, "pays": "TN", "statut": "OK"},
    {"id": 3, "montant": 300, "pays": "FR", "statut": "KO"},
    {"id": 4, "montant": 180, "pays": "TN", "statut": "OK"},
]

def est_valide(t):
    return t["montant"] > 100 and t["pays"] == "TN" and t["statut"] == "OK"

ids = [t["id"] for t in transactions if est_valide(t)]

print("Transactions validées (ids) :", ids)
```

15) Comment construire un dictionnaire de listes avec une compréhension ?

Créer le fichier **q15_dict_grouping.py** :

```
"""
```

Objectif

Regrouper des commandes par client.

Technique

On initialise d'abord les clés, puis on remplit.

La compréhension seule ne suffit pas toujours pour un grouping lisible.

```
"""
```

```
commandes = [
    {"id": 1, "client": "A", "total": 120},
    {"id": 2, "client": "B", "total": 80},
    {"id": 3, "client": "A", "total": 300},
]
```

```

        {"id": 4, "client": "C", "total": 150},
    ]

    clients = {c["client"] for c in commandes}
    groupes = {cl: [] for cl in clients}

    for c in commandes:
        groupes[c["client"]].append(c["id"])

    print("Groupes :", groupes)

```

16) Pourquoi `sum(gen)` est souvent meilleur que `sum(list(...))` ?

Créer le fichier `q16_sum_generateur.py` :

```

"""
Objectif
Sommer sans créer une liste intermédiaire.

Idée
sum accepte un itérable.
Un générateur évite d'allouer une liste en mémoire.
"""

n = 1_000_000

s1 = sum([i for i in range(n) if i % 2 == 0])
s2 = sum(i for i in range(n) if i % 2 == 0)

print("Somme avec liste :", s1)
print("Somme avec générateur :", s2)
print("Même résultat :", s1 == s2)

```

17) Comment créer un pipeline lisible avec générateurs (filtrer puis transformer) ?

Créer le fichier `q17_pipeline_generateurs.py` :

```

"""
Objectif
Créer un pipeline paresseux :
1) Filtrer

```

2) *Transformer*

3) *Consommer*

Intérêt

Très utile sur gros flux.

"""

```
def filtrer_vip(montants):  
    for m in montants:  
        if m > 100:  
            yield m
```

```
def appliquer_remise(montants):  
    for m in montants:  
        yield round(m * 0.9, 2)
```

```
ventes = [50, 120, 300, 80, 150, 40]
```

```
pipeline = appliquer_remise(filtrer_vip(ventes))
```

```
print("Résultats :")  
for x in pipeline:  
    print(x)
```

18) Pourquoi un générateur peut "cacher" une exception jusqu' à la consommation ?

Créer le fichier **q18_exception_tardive.py** :

"""

Objectif

Montrer qu'un générateur exécute le code au moment où on itère, pas avant.

Conséquence

Une erreur peut apparaître plus tard, au moment de next/for.

"""

```
def gen():  
    yield 1  
    yield 2  
    # Erreur volontaire : division par zéro  
    yield 10 / 0  
    yield 3
```

```

g = gen()

print(next(g))
print(next(g))

try:
    print(next(g))
except ZeroDivisionError as e:
    print("Erreur détectée à la consommation :", e)

```

19) Comment "redémarrer" un générateur proprement ?

Créer le fichier **q19_redemarrer_generateur.py** :

```

"""
Objectif
Redémarrer un parcours.

Principe
On ne redémarre pas un générateur épuisé.
On recrée la source (fonction génératrice) ou on stocke les données.
"""

def source():
    for i in range(5):
        yield i

g1 = source()
print("G1 :", list(g1))

g2 = source()
print("G2 :", list(g2))

```

20) Comment prédire la sortie d'une compréhension avec print dans le filtre ?

Créer le fichier **q20_predire_sortie.py** :

```

"""
Objectif
Comprendre l'ordre d'évaluation :
- range produit 0..4
- print(x) s'exécute dans la condition

```

*- print retourne None, donc condition toujours fausse
Résultat final : liste vide
Mais les prints s'affichent quand même.
"""*

```
res = [x for x in range(5) if print("test :", x)]  
print("Résultat final :", res)
```